

Implementation Description

Our system follows a microservice architecture which consists of two independent RESTful services, the authentication service and URL shortener service running on separate ports, enforcing separation of responsibilities.

Authentication Service

The authentication service manages users and authenticates based on JWT using an in memory dictionary of users, mapping usernames to passwords. Here the endpoints:

- Posts / users: creates a new user returning 201 or 409 “duplicate”
- Put /users: updates a password returning 200 or 403 “forbidden”
- Post /users/login: authenticates a user and returns a JWT with 200 or 403 “forbidden”

JWT Construction

JWTs are constructed without using a JWT library, and the token consists of:

Header {“alg”: “HS256”, “typ”: “JWT”}

Payload: {“sub”: <username>”}

The payload is minimal to reduce token size and hashing cost. Both the header and the payload are: JSON and base64url encoded, concatenated using “.”, and signed using HMAC-SHA256.

The secret key is stored in an environment variable “JWT_SECRET” and is only available to the authentication service. This ensures that the URL shortener service does not know the secret and that only the authentication service can generate valid tokens.

Token Validation

A dedicated endpoint (POST /validate is implemented in the authentication service, first the URL shortener sends the received token to this endpoint, if the token is valid then the endpoint returns the associated username, else it returns a 403.

URL Shortener Service

The URL shortener from assignment 1 has been extended to support multiple users. Now each URL mapping is stored as:

```
{
  Id: {
    "url": "<original_url>",
    "owner": "<username>"
  }
}
```

The owner is obtained from validating the JWT using the authentication service

Authentication Enforcement

The following endpoints require authentication:

- POST /
- GET /
- PUT /:id
- DELETE /:id
- DELETE /

If the user does not authenticate to the correct user or fails to authenticate, then the service returns a 403 “forbidden”. Only the GET /:id performs a 301 redirectg without requiring the user to authenticate.

Ownership Enforcement

For update and delete operations, the service simply checks that authenticated_user == mapping.owner, else a 403 is returned, making sure that users are only able to see their own shortened URLs.

Single Entry Point for All Microservices

Although all services run on separate ports, a production style architecture exposes them all through a single entry point by using a reverse proxy or a an API gateway like NGINX.

This gateway would:

- Listen on a single external port
- Have a route for
 - /auth/* to the authentication service
 - /short/* to the URL shortener service

With this approach, the internal service ports are hidden, allows TLS termination at one location, centralized logging and rate limiting, and simplifies the client interaction. This way the clients interact with a single public endpoint while the system remains internally modular.

Independent Scaling of Services

In a microservice architecture the services can be scaled independently based on user demand. If the URL shortener is experience high traffic due to frequent redirects or the authentication service is experience lower traffic because login is happening less frequently.

For independent scaling:

1. Containerize each service (maybe using Docker)
2. Deploy replicas of the overloaded service (either URL shortener or authentication service)
3. Place replicas behind a load balancer to ensure that users use different replicas based on demand

Alternatively, in a Kubernetes – based deployment:

1. Each service runs as a deployment
2. A Horizontal Pod Autoscaling (HPA) could increase or decrease the replicas based on CPU usage or rate of requests.

This allows one of the two services to be scaled without scaling the other, which would improve the resource efficiency. The in memory storage would be replaced by a shared datastore in order to support the horizontal scaling.

Managing a Distributed Microservice System

In order to manage distributed services we need observability and coordination.

Monitoring Health: Each service requires exposing a health endpoint via /health, and an orchestrator such as Kubernetes uses readiness and liveness requests in order to restart failed instances or avoid routing traffic to unhealthy services.

Metrics Collection: Collecting key metrics such as request rate, response latency, error rates, CPU and memory usage. These metrics can be exposed via a /metrics endpoint and could be collected by Prometheus, and Grafana could be used to visualize these metrics or generate alerts when thresholds are passed.

Logging: Centralized logging using ELK stack which could aggregate logs from all the different services, and enable debugging across the service boundaries as well.

Service Discovery: Instead of using hardcoded IP address, service discovery such as Kubernetes DNS, allows services to communicate with stable service names allowing dynamic scaling.

Altogether these mechanisms ensure reliable operation of a distributed microservice architecture.

Contribution Table

Winston Awortwi: Report writing

Tara Mac-Lean: Implemented authentication service

Alexandra Muntean: Improved URL shortening service